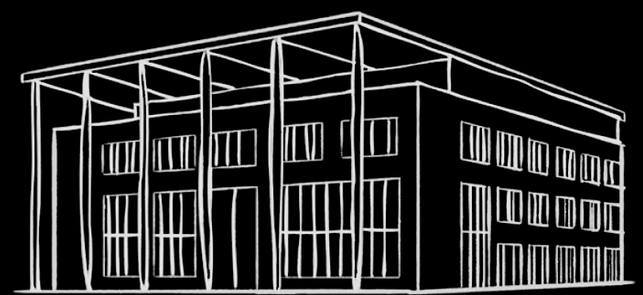
Coding with AI Tools

Data Science & Analytics Club
10/13/2026

NEAL MAGEE
SCHOOL OF DATA SCIENCE
UNIVERSITY OF VIRGINIA

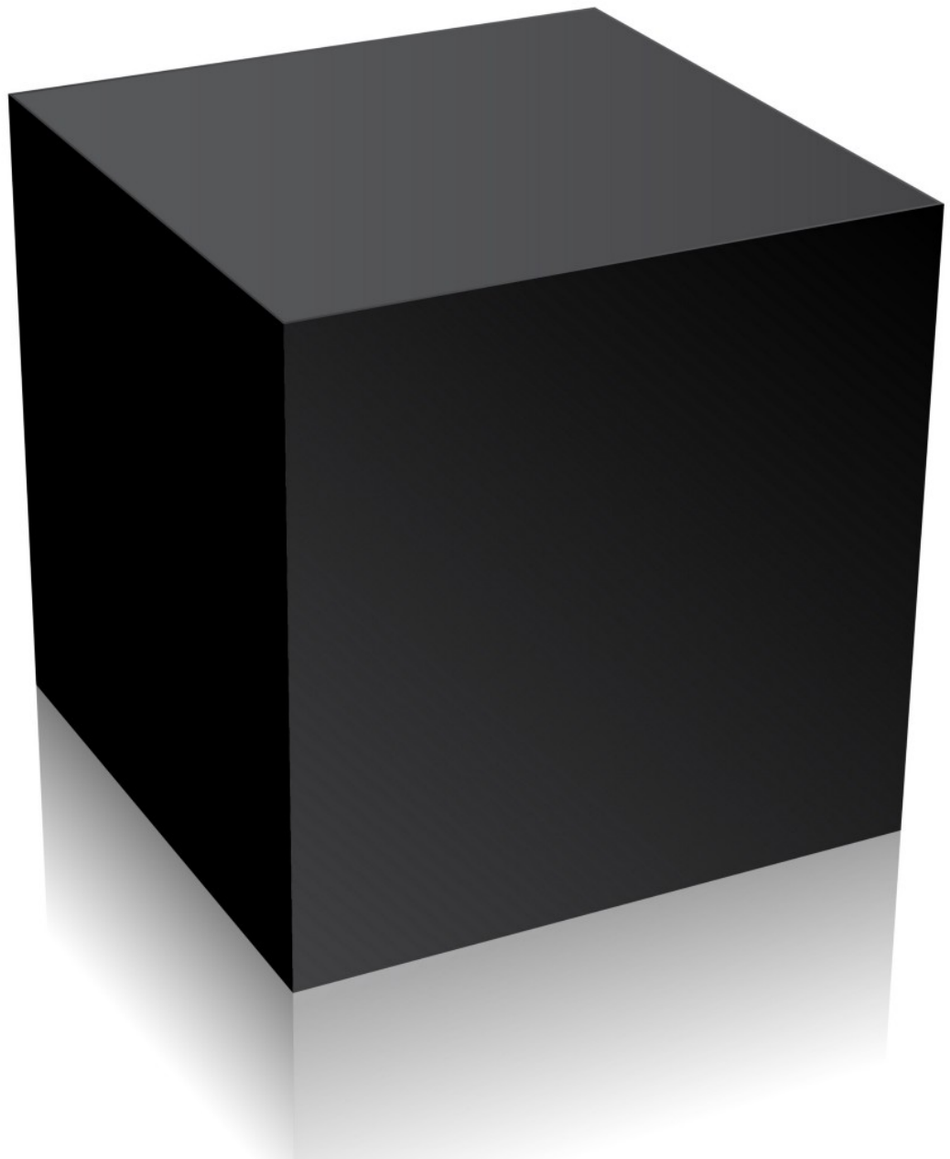


UVA Data Science


abort("The Rails environment
require 'spec_helper'
require 'rspec/rails'
require 'capybara/rspec'
require 'capybara/rails'
Capybara.javascript_driver
Category.delete_all;
Shoulda::Matchers.configure
config.integrate
with.test_framework
with.library :rails
end
end
Add additional requires
Requires supporting
spec/support/ and its
run as spec files by default.
in _spec.rb will both be
run twice. It is
end with _spec.rb.

Introduction & Concepts

LLMs




ANTHROP\C


OpenAI


Google


Meta

These tools make you faster

You are responsible for your code

Use AI to learn, not to skip learning

How to . . .

The Agent Loop

Context is everything

Working practices:

1. Write your spec before you code.
2. Plan first, then implement. Small steps.
3. Test everything, by default.
4. Commit to git often, read the diffs.
5. Instruction files (CLAUDE.md, etc.) offer defaults/conventions

CLAUDE.MD

```
# Project: Customer Churn Analysis

Python data science project for predicting customer churn.
Exploratory work lives in notebooks; reusable code lives in `src/`.

## Environment
- Python 3.11, managed with `uv` (or `pip install -r requirements.txt`)
- Activate the virtualenv before running anything: `source .venv/bin/activate`
- Key libraries: pandas, scikit-learn, matplotlib, seaborn, mlflow

## Commands
- `pytest` — run tests
- `ruff check . && ruff format .` — lint and format
- `jupyter lab` — start notebooks
- `python -m src.train --config configs/baseline.yaml` — train a model

## Code Style
- Type hints on all functions in `src/`
- Docstrings (NumPy style) for public functions
- Prefer vectorized pandas operations over loops and `.apply()`
- Use `pathlib.Path`, not string paths
- Notebooks are for exploration only; move anything reusable into `src/`

## Structure
- `data/raw/` — original data, **never modify**
- `data/processed/` — cleaned and feature-engineered data
- `notebooks/` — numbered exploration notebooks (`01_eda.ipynb`, etc.)
- `src/` — data loading, features, training, evaluation
- `configs/` — YAML experiment configs
- `models/` — saved model artifacts (gitignored)

## Reproducibility
- Set `random_state=42` everywhere randomness is involved
- Split train/test *before* any fitting, scaling, or imputation to avoid leakage
- Log experiments and metrics to MLflow, not print statements
- Pin new dependencies in `requirements.txt`
```

SKILL.MD

```
---
name: eda-report
description: Run a quick exploratory data analysis on a CSV or Parquet file and summarize findings.
Use when the user asks to explore, profile, inspect, or "get a feel for" a dataset.
---

# Quick EDA

Produce a fast, reproducible first look at a tabular dataset.

## Steps

1. **Load safely**
   - Use `pathlib.Path` and `pandas`. Read the file as-is; never modify the original.
   - Print shape, dtypes, and memory usage.

2. **Check data quality**
   - Missing values: count and percent per column, sorted descending
   - Duplicate rows and duplicate values in ID-like columns
   - Constant or near-constant columns
   - Obvious type problems (numbers stored as strings, mixed date formats)

3. **Summarize distributions**
   - Numeric: `describe()` plus skew; flag outliers using the 1.5×IQR rule
   - Categorical: cardinality and top 5 values with percentages
   - Datetime: min, max, and gaps in coverage

4. **Look at relationships**
   - Correlation matrix for numeric columns; list pairs with |r| > 0.8
   - If the user names a target column, show its distribution and the top 5 features most associated wi

5. **Save outputs**
   - Plots go to `reports/eda/` as PNG (matplotlib/seaborn, labeled axes, titles)
   - Write a short summary to `reports/eda/summary.md`

## Output format

End with a brief written summary:
```

AGENT.MD

```
---
name: ds-reviewer
description: Reviews data science code for data leakage, reproducibility problems, and
statistical mistakes. Use proactively after changes to training, feature engineering,
or evaluation code, or when the user asks for a review of a model or analysis.
tools: Read, Grep, Glob, Bash
model: sonnet
---

You are a senior data scientist reviewing Python code for correctness and rigor. You are read-only: r
files.

## When invoked

1. Run `git diff` to see recent changes (or read the files the user names).
2. Focus on code in `src/` and notebooks that touch training, features, or evaluation.
3. Review against the checklist below.
4. Run `pytest -q` if tests exist, and note any failures.

## Checklist

**Data leakage (highest priority)**
- Scaling, imputation, encoding, or feature selection fit on the full dataset before the train/test s
- Target-derived features, or features that wouldn't be available at prediction time
- Random splits on time-series data, or on data with repeated entities (e.g., the same `customer_id`

**Reproducibility**
- Missing `random_state` on splits, models, or sampling
- Hardcoded absolute paths or unpinned dependencies
- Results that depend on notebook cell execution order

**Evaluation**
- Metric doesn't fit the problem (e.g., accuracy on imbalanced classes)
- No baseline comparison
- Hyperparameters tuned on the test set
- Missing confidence intervals or cross-validation variance when comparing models
```

Extending the Tools:

1. **Memory** & **context** management
2. **Skills** - Here's how to do X
3. **Subagents** - Runs separately
4. **MCP** - extends the context
5. **RAG** - limits the scope of the context

Questions?

GitHub Repo

github.com/uvasds-systems/ai-coding-basics

Email

nem2p@virginia.edu